

# From Vibe Coding to Agentic Engineering (agy & claude)

The Prerequisite Engineering Course Preceding Spec-Driven Development (SDD-Crash-Course)

## 1. Code as Model & Language as Action

Fowler (May 2026) & Wittgenstein’s 2 Theories:

- Code is not merely syntax for a machine; it is an explicit **model of understanding**.
- Early *Tractatus* (Picture Theory / formal code) vs. Late *Investigations* (Language as Action).
- Shared domain vocabulary + coworker clarification gates (Lab 01).

## 2. Context, Memory & Curated Skills

Progressive Disclosure & SkillsBench:

- Concise root context ( $\leq 40$  lines) with on-demand @docs/\*.md imports + Git-worktree Auto-Memory (Lab 02).
- Craft agentskills.io skills manually (+16.6 pp); never self-generate (−8.1 to −11.5 pp) (Lab 03).
- Cap active skills at 2-3 & calibrate triggers with eval-viewer + Signal Detection Theory (Lab 04).

## 3. Multi-Agent, Plugins & SDD Bridge

feature-dev, MCP & Harness Flywheel:

- Orchestrate read-only code-explorer, code-architect, and code-reviewer subagents (Lab 05).
- Bundle skills, subagents, PreToolUse hooks, and MCP servers into versioned plugins (Lab 06).
- Escalate  $\geq 3$ -service brownfield changes to **Spec-Driven Development** (SDD-Crash-Course).

## Dual-Branch Parity (agy for Antigravity / Gemini CLI × claude for Claude Code) + 6 Self-Diagnosing Python Labs

- **Module 01:** Foundations, Fowler, Wittgenstein’s 2 Theories & Dual Harnesses
- **Module 02 (Lab 01):** Shared Vocabulary & Clarification Gates
- **Module 03 (Lab 02):** CLAUDE.md/GEMINI.md & Worktree Memory
- **Module 04 (Lab 03):** Crafting anthropic-brand/ Skills Manually
- **Module 05 (Lab 04):** eval-viewer & Skill Overload
- **Module 06 (Lab 05):** 7-Phase feature-dev Multi-Agent
- **Module 07 (Lab 06):** Plugins, MCP & SDD Bridge

# Unmesh Joshi's "What Is Code?" (May 2026): Code as a Model of Understanding

Why cheaper LLM syntax makes domain vocabulary, explicit conceptual models, and active thinking more important than ever

## The Trap: "Ralph Wiggum" Vibe Coding & Cognitive Debt

- 1. Passive Diff Review:** Treating the LLM as a black-box syntax generator and judging output only by whether a demo runs.
- 2. Vocabulary & Synonym Drift:** Across three prompts, the agent invents `client_id`, `cust_acct`, and `user_id` for the same account entity.
- 3. IEEE-754 Float Corruption:** Mixing `tx_amt = 19.99` floats instead of integer minor units (`MoneyCents`), breaking conservation.
- 4. Silent Hallucination:** Guessing missing exchange rates (`rate = 1.0`) instead of asking a coworker-style clarifying question.

## The Solution: Explicit Conceptual Models & Vocabulary

- 1. Unmesh Joshi's Thesis (martinfowler.com):** "We are not meant to be passive reviewers of generated code. The act of writing code is itself part of our thinking."
- 2. Ubiquitous Language Value Objects:** Encode domain nouns and invariants directly into frozen types (`AccountId`, `MoneyCents`, `SettlementBatch`).
- 3. Iterative Vocabulary Refinement:** Refine domain terms through feedback, domain expertise, and deterministic tests.
- 4. Foundation for Humans + LLMs:** Clean abstractions compress context so both humans and agents reason accurately.

## Core Principle: Code Is Still Instructions for a Machine—But Primarily a Model of Understanding

- "As LLMs make code generation cheaper, the mechanical act of writing instructions becomes less central. What becomes more important is **making the conceptual model explicit, discovering the right vocabulary, and refining that vocabulary through iteration, domain expertise, and feedback.**" — *Unmesh Joshi, martinfowler.com, May 2026*

# Wittgenstein's Two Theories of Language: How Language Produces Actions & Code

Why Vibe Coding requires both Early Wittgenstein's formal precision and Late Wittgenstein's collaborative language-as-action

## 1. Early Theory: Picture Theory

*Tractatus Logico-Philosophicus* (1921):

- Language as a strict, formal, 1-to-1 logical picture of facts in the world.
- **Software Analog:** Programming languages, compilers, type systems (`AccountId`, `MoneyCents`), and unit tests.
- Gives the machine an unambiguous formal specification.

## 2. Late Theory: Meaning as Use

*Philosophical Investigations* (1953):

- Words are tools in a collaborative **Language Game** (*Builder & Assistant*, §2, §43).
- **Vibe Coding Analog:** Using natural language with an AI coworker inside a shared context (`CLAUDE.md/GEMINI.md`, Skills) to coordinate actions and produce code (`arXiv:2302.01570`).

## 3. Empirical Proof in Coding Agents

Stanford `SHRDLURN` (`arXiv:1606.02447`):

- 100 players instructing an AI builder via language: success depends on **avoiding synonyms & reusable abstractions** (Skills).
- **MaKTO** (`arXiv:2501.14225`) & **LGDL**: Grounded dialogue wins **61% (+23% vs. GPT-4o)**; asks clarifying questions when `conf < 0.85`.

## The Vibe Coding Synthesis: Late Wittgenstein Drives the Conversation → Early Wittgenstein Verifies the Code

- We use **Late Wittgenstein's collaborative language-as-action** to align intent, vocabulary (`CLAUDE.md/GEMINI.md`), and skills (`SKILL.md`) with our AI coworker—so the agent produces **Early Wittgenstein's formal code** verified by types and tests (*Winograd & Flores, 1986*).

# Humans and Agents in Software Engineering Loops (Kief Morris, martinfowler.com, Mar 2026)

Separating the human-owned “Why Loop” from the agent-executed nested “How Loops” via Harness Engineering

## 1. Human “Why Loop”

### Problem & Value Framing:

- Define business outcomes, domain boundaries, and Ubiquitous Language.
- Own the conceptual model and ethical/architectural trade-offs.
- Never abdicate the *Why* to statistical token generation.

## 2. Outer “How Loop”

### Context & Memory Harness:

- Root `CLAUDE.md` / `GEMINI.md` ( $\leq 40$  lines) + `@docs/*.md` imports.
- Worktree-shared Auto-Memory (`MEMORY.md` / `KNOWLEDGE.md`) with `#direct` provenance.

## 3. Middle “How Loop”

### Skills & Subagents:

- Manually crafted `SKILL.md` manuals (2–3 active skills sweet spot).
- 7-phase `feature-dev` orchestration (`code-explorer`, `code-architect`, `code-reviewer`).

## 4. Inner “How Loop”

### TDD & Immutable Verifier:

- Fast code generation verified by deterministic unit tests.
- Protected by `PreToolUse` hooks blocking edits to `adversarial_tests/`.

## Kief Morris’s “On-the-Loop” Harness Flywheel (Implemented in Lab 06 `harness_flywheel.py`)

- **In-the-Loop (Bottleneck):** Human manually fixes every broken code line in the Inner How Loop.
- **On-the-Loop (Flywheel):** Whenever an artifact defect escapes, upgrade the **Harness** (`ROOT_CONTEXT_RULE`, `AUTO_MEMORY`, `CURATED_SKILL`, or `PRE_TOOL_USE_HOOK`) so that entire class of errors never recurs.

# 1-to-1 Architectural Parity: c1aude Branch vs. agy Branch

Master portable engineering principles once and execute natively in both Claude Code and Antigravity / Gemini CLI

## c1aude Branch (Claude Code Harness)

- **Persistent Context:** `CLAUDE.md` ( $\leq 40$  lines) with `@docs/*.md` imports + `.claude/rules/*.md` (`paths: globs`).
- **Auto-Memory:** `~/.claude/projects/<git-repo>/memory/MEMORY.md` (shared across git worktrees; first 200 lines / 25 KB auto-loaded).
- **Skills & Evaluation:** `.claude/skills/anthropic-brand/SKILL.md` invoked via `/anthropic-brand`; evaluated with `eval-viewer`.
- **Multi-Agent & Plugins:** `.claude/agents/{code-explorer, code-architect, code-reviewer}.md` + `.claude-plugin/plugin.json` & `.mcp.json`.
- **Session Commands:** `/context`, `/memory`, `/feature-dev`, `/compact`, `/clear`.

## agy Branch (Antigravity / Gemini CLI Harness)

- **Persistent Context:** `GEMINI.md` & `AGENTS.md` ( $\leq 40$  lines) with `@docs/*.md` imports + `.agents/rules/*.md` (`always_on | model_decision`).
- **Knowledge Base:** `~/.gemini/antigravity/knowledge/<project>/KNOWLEDGE.md` + `#direct`, `#commit`, `#time`, `#session` tags.
- **Skills & Evaluation:** `.agents/skills/anthropic-brand/SKILL.md`; evaluated with paired `eval-viewer` & SDT *d'*.
- **Multi-Agent & Plugins:** `.agents/agents/*.md` + `.agents/workflows/feature-dev.md` + `.agents/plugins/vibe-engineering-kit/plugin.json`.
- **Session Commands:** `/stats`, `/memory` (`show | add | refresh`), `/skills`, `/plan`, `/compress`.

## Verified by Automated Track Suites (`tests/claude_track/` & `tests/agy_track/`)

- Both branches run identical domain labs (`labs/lab_01..06`) plus dedicated harness contract test suites (`CI=true python3 -m unittest discover -s tests -p "test_*.py" -v`).

# The 6 Self-Diagnosing Python Labs (`./self_diagnose_all.sh` in $< 1s$ )

Every lab includes a broken vibe-coded `starter/`, a verified `solution/`, and read-only `adversarial_tests/`

## Lab 01: Domain Vocabulary

`domain_ledger.py` (REQ-0101..0106)

- `AccountId` & `MoneyCents` value objects.
- Double-entry `SettlementBatch`.
- 3-way coworker clarification gate.

## Lab 02: Context & Memory

`context_memory_manager.py` (REQ-0201..0206)

- $\leq 40$ -line root context + `@docs/*.md`.
- Git-worktree Auto-Memory path resolver.
- `#direct` provenance & 200-line cap.

## Lab 03: Crafting Skills

`skill_validator.py` (REQ-0301..0306)

- `agentskills.io` frontmatter validator.
- `anthropic-brand/` 4-file anatomy.
- Rejects self-generated filler phrases.

## Lab 04: eval-viewer & Overload

`skill_eval_engine.py` (REQ-0401..0406)

- Paired `with_skill` vs. `without_skill`.
- 2-3 skill cap & Reference-First merge.
- Trigger calibration & `eval-viewer` HTML.

## Lab 05: Multi-Agent feature-dev

`feature_dev_orchestrator.py` (REQ-0501..0506)

- Parallel `code-explorer` (`file:line`).
- Phase 3 `ClarificationGateError` STOP.
- Tri-lens `code-architect` &  $\geq 80$  review.

## Lab 06: Plugins, MCP & SDD

`harness_flywheel.py` (REQ-0601..0606)

- `plugin.json` + JSON-RPC MCP tool.
- `PreToolUse` immutable test guard.
- Flywheel router  $\rightarrow$  `ESCALATE_TO_SDD_SPEC`.

# Lab 01 Problem: The “Vibe-Coded” Payments Ledger Disaster

Inspecting `labs/lab_01_vocabulary_and_bounded_language_games/starter/domain_ledger.py`

## What the Ungrounded “Vibe-Coded” Starter Does Wrong

### 1. Synonym Soup (`starter/domain_ledger.py`):

- Accepts arbitrary dict keys (`acct`, `client_id`, `cust_acct`, `user_id`) with zero regex or type validation.

### 2. IEEE-754 Float Drift:

- Uses `float(entry.get("tx_amt", 0.0))`, accumulating `0.1 + 0.2 != 0.3` penny errors across settlement batches.

### 3. Silent FX Hallucination:

- When asked to settle `JPY -> USD` without a grounded FX rate, it silently invents `rate = 1.0` and executes!

## Target Architecture: Shared Vocabulary & Clarification Gate

### 1. Ubiquitous Language Value Objects (`REQ-0101..0102`):

- Frozen `AccountId` (`^ACCT-[A-Z0-9]{4,12}$`) and `MoneyCents` (`amount_cents: int, currency: str`).

### 2. Double-Entry Conservation (`REQ-0103`):

- `SettlementBatch.apply_postings()` enforces  $\sum \text{debits} = \sum \text{credits}$  and non-negative balances.

### 3. Coworker Clarification Gate (`REQ-0104..0105`):

- `evaluate_request_readiness()` returns `CLARIFICATION_REQUIRED` when FX rates or vocabulary are ungrounded.

## Terminal Command to Witness the Starter Failure vs. Solution Pass

- `./labs/lab_01_vocabulary_and_bounded_language_games/self_diagnose.sh labs/lab_01_vocabulary_and_bounded_language_games/starter` → **FAILS** on float & FX checks!

# Lab 01 Step 1 (REQ-0101..0102): Ubiquitous Language Value Objects

Replacing untyped dicts and IEEE-754 floats with frozen `AccountId` and `MoneyCents` dataclasses

## AccountId & MoneyCents Invariants (REQ-0101)

- **AccountId(value: str):** Frozen dataclass validating `^ACCT-[A-Z0-9]{4,12}$`. Rejects drift synonyms (`cust_acct_99`) with `DomainVocabularyError`.
- **MoneyCents(amount\_cents: int, currency: str):** Rejects `float` and `bool` with `TypeError` (`isinstance(amount_cents, bool)` or `not isinstance(amount_cents, int)`).
- **Supported Currencies:** Validates `currency` in `{"USD", "EUR", "GBP", "JPY"}` and enforces currency match on `add()` / `subtract()`.

## Deterministic Basis-Point Allocation (REQ-0102)

- **Why Floats Fail in Fee Calculation:** Computing `round(10005 * 0.0029)` in floating point introduces representation drift.
- **Half-Up Integer Arithmetic:** `allocate_fee_bps(fee_bps: int)` computes:  

```
numerator = self.amount_cents * fee_bps  
fee_cents = (numerator + 5000) // 10000
```
- **Returns** `(net_money, fee_money)` where `net_money.add(fee_money) == self` holds as an exact mathematical identity.

## Fowler Takeaway: The Type System Is Your Shared Vocabulary Contract

- Once `AccountId` and `MoneyCents` are referenced in `CLAUDE.md` / `GEMINI.md`, every subsequent prompt uses the exact same domain nouns and integer arithmetic.

# Lab 01 Step 2 (REQ-0103): Bounded-Context Double-Entry Conservation

Enforcing strict conservation of money and non-negative balances in `SettlementBatch.apply_postings()`

## LedgerPosting & SettlementBatch Contract

- **LedgerPosting:** Pairs `debit_account: AccountId`, `credit_account: AccountId`, `amount: MoneyCents`, and a non-empty `narrative: str`.
- **Self-Transfer Guard:** Rejects `debit_account == credit_account` and `amount.amount_cents <= 0` with `LedgerConservationError`.
- **Currency Homogeneity:** Every posting in a `SettlementBatch` must match the batch's base currency before any balance is mutated.

## Atomic Conservation Verification (REQ-0103)

- **Pre-Execution Snapshot:** Computes `before_total = sum(b.amount_cents for b in updated.values())`.
- **Overdraft Prevention:** If any debit exceeds the source account's current balance, raises `LedgerConservationError` without mutating caller state.
- **Post-Execution Invariant:** Verifies `after_total == before_total` ( $\sum \text{debits} = \sum \text{credits}$ ), returning a brand-new immutable balance map.

## Wittgensteinian Lesson: Situated Rules Define Valid Moves in the Game

- Just as chess rules forbid moving a rook diagonally, `SettlementBatch` makes an unbalanced or overdraft settlement move unrepresentable at runtime.

# Lab 01 Step 3 (REQ-0104..0105): Coworker Readiness & Clarification Gate

Evaluating settlement instructions in `evaluate_request_readiness()` (aliased as `evaluate_language_game_move()`)

## 1. ESCALATE\_OUT\_OF\_BOUNDS

### Policy Violation (conf = 0.0):

- Triggered when a prompt attempts a prohibited move (`override_audit`, `bypass_conservation`, `negative_settlement`).
- Immediately halts and escalates to a human operator.

## 2. CLARIFICATION\_REQUIRED

### Ambiguous Request (conf < 0.85):

- Penalties for drift terms (`client_id`, `tx_amt`, `fee_float`), float amounts, or missing (`src`, `dst`) FX rate in `known_fx_basis_points`.
- Returns explicit coworker-style `questions` instead of guessing!

## 3. GROUNDED\_EXECUTE

### Grounded Request (conf ≥ 0.85):

- Canonical `AccountIds`, integer `amount_cents`, and verified currency/FX rate.
- Safely constructs and returns the normalized `LedgerPosting`.

## Coworker Clarification Invariant (REQ-0105): Never Invent Missing FX Rates or Business Policy

- When `source_currency != target_currency` and (`src`, `dst`) is missing from `known_fx_basis_points`, confidence drops below 0.85 and the agent asks for the authoritative basis-point rate.

# Lab 01 Step 4 (REQ-0106): Self-Diagnosis & Adversarial Verification

Verifying `domain_ledger.py` against unit tests and locked `adversarial_tests/`

## Running Lab 01 Self-Diagnosis (`self_diagnose.sh`)

- **Command:**

`./labs/lab_01_vocabulary_and_bounded_language_games/self_diagnose.sh`

- **Two-Stage Verification Gate:**

1. Runs `solution/test_domain_ledger.py` (6 unit tests covering REQ-0101..0106).
  2. Runs `adversarial_tests/test_domain_ledger_verifier.py` (3 Clean-Room adversarial tests).
- Executes in `< 0.02s` using Python's standard library only.

## What the Locked Adversarial Verifier Catches

- **Boolean Subtype Trap:** In Python, `isinstance(True, int)` is `True`! The adversarial verifier checks that `MoneyCents(True, "USD")` raises `TypeError`.
- **Mutating Input Dicts:** Verifies that a failed `SettlementBatch.apply_postings()` call leaves the caller's original `balances` dictionary 100% untouched.
- **Missing Cross-Currency FX:** Verifies that `JPY -> USD` without an FX entry returns `CLARIFICATION_REQUIRED`.

## Module 02 Complete → Next: Persistent Context (`CLAUDE.md` / `GEMINI.md`) & Cross-Session Auto-Memory (Lab 02)

- Now that our domain vocabulary is encoded in types, how do we persist project rules and agent learnings across fresh context windows without bloating startup tokens?

# Lab 02 Problem: Fresh Context Windows, Monolithic Bloat & Memory Poisoning

Inspecting `labs/lab_02_persistent_context_and_memory_hygiene/starter/context_memory_manager.py`

## Three Context & Memory Failure Modes in the Starter

### 1. 500-Line Monolithic `CLAUDE.md` / `GEMINI.md`:

- Dumping every schema, runbook, and historical postmortem into the root file wastes thousands of startup tokens on every turn.

### 2. Worktree Memory Fragmentation:

- Using `os.getcwd()` instead of the git repository root creates disconnected memory silos across git worktrees and subdirectories.

### 3. Provenance Poisoning (`#direct` Abuse):

- Tagging unverified agent hypotheses with `#direct` turns transient guesses into permanent rules.

## Two Mechanisms Carry Knowledge Across Fresh Windows

### 1. Persistent Project Context (Human-Written):

- Concise  $\leq$  40-line `CLAUDE.md` (`claude`) or `GEMINI.md/AGENTS.md` (`agy`) with `@docs/*.md` on-demand references.

### 2. Cross-Session Auto-Memory (Agent-Written):

- Git-worktree-shared `~/.claude/projects/<project>/memory/MEMORY.md` (`claude`, capped at **200 lines / 25 KB**) or per-project `~/.gemini/antigravity/knowledge/<project>/KNOWLEDGE.md` (`agy`).

## The Fundamental Equation of Context Engineering: $\text{Output} = f(\text{Context})$

- Model reasoning quality peaks at **40%–60% context utilization** and degrades past **60%**. Startup context + memory must consume  $< 5\%$  of the context window.

# Lab 02 Step 1 (REQ-0201..0202): Concise Root Context & @path Imports

Auditing root CLAUDE.md / GEMINI.md files and loading @docs/\*.md references on demand

## Auditing Root Context (audit\_root\_context\_file)

- **Hard Line Budget (REQ-0201):** Enforces `line_count <= 40` (configured via `MAX_ROOT_CONTEXT_LINES = 40`).
- **3 Required Onboarding Sections:**
  1. **Commands & Environment** (exact test/build commands).
  2. **Architecture & Directory Map** (high-level package layout).
  3. **Core Conventions** (non-obvious project invariants).
- Extracts all `@relative/path.md` tokens via regex `r"(?!\\w)@([a-zA-Z0-9_./-]+.md)"`.

## Progressive @path Loading (resolve\_on\_demand\_references)

- **Selective Expansion (REQ-0202):** Takes `active_topics: set[str]` and resolves only matching `@docs/<topic>.md` files.
- **Path-Traversal Security Guard:** Resolves `(project_root / rel_path).resolve()` and verifies `candidate.relative_to(root)` so malicious `@../../etc/passwd.md` imports raise `ContextHygieneError`.
- Saves > 80% of startup context tokens compared to inlining all docs.

## Key Analogy: @docs/\*.md inside CLAUDE.md/GEMINI.md Works Just Like Progressive Disclosure in Skills

- Keep the root file as a high-signal index ( $\leq 40$  lines); pull `@docs/architecture.md` or `@docs/testing-conventions.md` only when the task touches those subsystems.

# Lab 02 Step 2 (REQ-0203): Git-Worktree Auto-Memory Resolution

Why all subdirectories and git worktrees in the same repository share one unified Auto-Memory directory

## Claude Code Memory Path (`derive_claude_memory_dir`)

- **Rule:** Each project gets its own memory directory at `~/.claude/projects/<project>/memory/`.
- **Git Worktree Sharing:** When `git_common_root` is provided, all worktrees (`/worktrees/feat-a/src/api`, `/worktrees/bugfix-b/tests`) resolve to the **exact same canonical git root** before slugifying `/`  $\rightarrow$  `-`.
- **Fallback Outside Git:** Outside a git repository, `working_dir.resolve()` is used as the project root instead.

## Antigravity Knowledge Base (`KNOWLEDGE.md`)

- **Workspace Knowledge Base:** Persisted per project at `~/.gemini/antigravity/knowledge/<project>/KNOWLEDGE.md` with YAML frontmatter `scope: workspace`.
- **Structured Knowledge Items:** `topics/<domain>.md` and `projects/<name>.md` store cross-session architectural and debugging insights.
- **Session Memory Commands:** Inspect and refresh active workspace memory via `/memory show`, `/memory add`, and `/memory refresh`.

## Why Worktree Sharing Matters for Parallel Agent Engineering

- When you run 3 parallel agent sessions in separate git worktrees (`git worktree add`), lessons learned in one worktree are immediately available to the others.

# Lab 02 Step 3 (REQ-0204 . . 0205): Provenance Tags & The 200-Line Memory Cap

Preventing memory poisoning with `#direct` provenance validation and overflow topic routing

## Epistemic Provenance Discipline (REQ-0204)

- **The `#direct` Rule:** Reserve `#direct` *strictly* for facts, instructions, or preferences explicitly stated by the user (`is_explicit_user_statement=True`).
- **Indirect Observations:** Facts discovered by reading code, logs, or tests **MUST** omit `#direct` and instead carry relativity anchors:
  - `#commit:<git_sha>` (repository commit anchor)
  - `#time:<YYYY-MM-DD>` (wall-clock anchor)
  - `#session:<session_id>` (session transcript link).

## The 200-Line / 25 KB MEMORY.md Cap (REQ-0205)

- **Why Cap `MEMORY.md`:** Claude Code auto-loads only the first **200 lines / 25 KB** of `MEMORY.md` at session start.
- **`compile_memory_index()`:** Places `priority=True` entries in `MEMORY.md` up to `max_index_lines` (200) and `max_bytes` (25,000).
- **Topic Overflow Routing:** Routes non-priority or overflow notes into `topics/<topic>.md` and appends markdown links (– `[debugging] (topics/debugging.md)`) to `MEMORY.md`.

## Security Guard: `validate_and_format_memory_entry()` Rejects Spoofed `#direct` Tags

- Even if an agent passes `"#direct"` inside raw `text` when `is_explicit_user_statement=False`, Lab 02 raises `MemoryProvenanceError`.

# Lab 02 Step 4 (REQ-0206): /context, /memory, /clear & Verification

Inspecting live context utilization and verifying `context_memory_manager.py`

## Interactive Context & Memory Commands

- **/context (claude) & /stats (agy):** Display live token usage across system prompt, `CLAUDE.md/GEMINI.md`, memory, MCP tools, and skills.
- **/memory (claude & agy):** Inspects loaded memory files (`/memory show | add | refresh` in Gemini CLI) and prunes stale cross-session notes.
- **/clear & /compact (/compress):** Reset between unrelated tasks or compact history before hitting the 60% degradation wall.

## Running Lab 02 Self-Diagnosis (`self_diagnose.sh`)

- **Command:**  
`./labs/lab_02_persistent_context_and_memory_hygiene/self_diagnose.sh`
- **Verified Invariants (REQ-0201..0206):**
  - Bloated > 40-line root context flagged.
  - Path-traversal `@../secret.md` blocked.
  - Worktrees share identical `MEMORY.md` path.
  - Spoofed `#direct` tags rejected; overflow routed to `topics/*.md`.

## Module 03 Complete → Next: Crafting `agentskills.io` Skills Manually vs. Self-Generated Traps (Lab 03)

- How do we package domain workflows into reusable instruction manuals (`SKILL.md`) that lift pass rates by **+16.6 pp** without polluting the context window?

# Lab 03 Problem & Empirical Proof (SkillsBench arXiv:2602.12670)

Why manually written skills (“a conciencia”) boost pass rates by +16.6 pp while self-generated skills hurt performance

## Curated Human-Authored Skills: +16.6 pp Average Lift

SkillsBench (87 Tasks, 8 Domains, 9,396 Trajectories):

- Gemini CLI + Gemini 3.1 Pro: 36.0% → 60.8% (+24.8 pp).
- Claude Code + Opus 4.7: 43.0% → 61.2% (+18.2 pp).
- Codex CLI + GPT-5.5: 46.8% → 62.9% (+16.1 pp).
- Procedural Tasks: Up to +92.6 pp lift (llm-prefix-cache-replay: 1.9% → 94.4%) when encoding non-obvious procedural gotchas.

## Self-Generated Skills: −8.1 to −11.5 pp Degradation!

When the LLM Generates Its Own Skill Before Solving:

- Claude Code + Opus 4.7: 43.0% → 34.9% (−8.1 pp).
- Codex CLI + GPT-5.5: 46.8% → 35.5% (−11.3 pp).
- Gemini CLI + Gemini 3.1 Pro: 36.0% → 24.5% (−11.5 pp).
- Root Cause: Self-generated skills either repeat generic training-data platitudes (“write clean code”) or lock in hallucinated rules (e.g. 3d-scan-calc ×1000 unit bug).

## The Golden Law of Skill Authoring: “Experience Before Theory” & “Signal Not Noise”

- Never ask an LLM to generate a skill zero-shot. Run the agent *without* a skill first, observe where it fails, and manually write only the **Delta** from baseline.

# Lab 03 Step 1 (REQ-0301..0302): `agentskills.io` Spec & Progressive Disclosure

Enforcing strict YAML frontmatter rules and 3-level progressive disclosure in `skill_validator.py`

## Level 1: Startup Metadata

≈ 100 Tokens at Session Start:

- `name`: 1..64 chars matching `^[a-z0-9]+(-[a-z0-9]+)*$` and parent folder name.
- `description`: 1..1024 chars with positive (`Use when...`) and negative (`Don't use for...`) triggers.

## Level 2: `SKILL.md` Router

Loaded Only on Activation:

- Invoked manually via `/skillname` (e.g. `/anthropic-brand`) or auto-matched by description.
- Body kept between **50 and 500 lines** (< 5,000 tokens) with core invariants + routing table.

## Level 3: Companion Files

0 Tokens Until Needed:

- Artifact-specific markdown files (`docs.md`, `slides-deck.md`, `apply_template.md`).
- Loaded one level deep *only* for the requested artifact type (> 75% token savings).

## Validation Invariant (REQ-0301..0302): Why Description Must State Both Positive & Negative Triggers

- Without `Use when...` and `Don't use for...` clauses in `description`, the router either misses relevant tasks or false-alarms on unrelated prompts.

# Lab 03 Step 2 (REQ-0303..0304): The Multi-File anthropic-brand/ Skill

Inspecting the 4-file progressive disclosure directory in `.claude/skills/anthropic-brand/` and `.agents/skills/anthropic-brand/`

## `anthropic-brand/SKILL.md` (Level 2 Core Router)

- **Core Palette Tokens:** Warm Ivory #FAF9F5, Charcoal Slate #141413, Terracotta #D97757, Muted Stone #5E5D59.
- **Experience-Grounded Gotcha:** Never pair Terracotta (#D97757) text directly on Muted Stone (#5E5D59) backgrounds (contrast  $2.1 : 1 < 4.5 : 1$  WCAG AA minimum).
- **Selective Routing Table:** Directs the agent to read *only* the companion file matching the target deliverable.

## Level 3 Companion Files (Loaded on Demand)

- **1. `docs.md`:** Loaded only for RFCs, Markdown docs, or LaTeX/PDF reports (72ch measure, callout borders, table formatting).
- **2. `slides-deck.md`:** Loaded only for 16:9 widescreen presentations (1920 × 1080 grid, ≤ 3 cards/row, 28pt title minimum).
- **3. `apply_template.md`:** Loaded when retrofitting an existing draft (4-step audit, token replacement, contrast verification).

## Why One-Level-Deep Links Matter (REQ-0304)

- `SKILL.md` links directly to `docs.md`, `slides-deck.md`, and `apply_template.md`. Never chain `SKILL.md -> a.md -> b.md -> c.md` (agents lose track of multi-hop links).

# Lab 03 Step 3 (REQ-0305): Rejecting Self-Generated Filler vs. High-Signal Rules

How `audit_skill_signal_quality()` detects low-signal LLM-generated skills

## Rejected: Self-Generated Filler (`bad_self_generated_skill/`)

- **YAML Violations:** `name: Bad_Self_Gen_Skill` (uppercase + underscores), vague 5-word description with no `Use when/Don't use for.`
- **Training-Data Platitudes Flagged by `SELF_GEN_FILLER_PHRASES`:**
  - “write clean, modular, maintainable code”
  - “follow industry best practices and design patterns”
  - “make sure there are no bugs”
  - “handle edge cases appropriately”.
- Zero concrete hex codes, commands, or failure gotchas!

## Approved: Manually Crafted High-Signal Skill (REQ-0305)

- **High-Signal Density Checks in `audit_skill_signal_quality()`:**
  1. **Line Count Gate:**  $35 \leq \text{lines} \leq 500$  (neither a stub nor an encyclopedia).
  2. **Zero Filler Phrases:** 0 matches against generic pretraining platitudes.
  3. **Concrete Prescriptive Artifacts:**  $\geq 3$  concrete hex tokens, file paths, or fenced commands.
  4. **Grounded Gotchas:** Explicit `Gotcha/Trap/Never` failure-mode section.

## Anthropic & Google `skill-creator` Rule: Explain the “Why”, Not Just Rigid “ALWAYS/NEVER” Screaming

- Explaining *why* a rule exists (e.g. WCAG 2.1 : 1 contrast failure) lets the model generalize intelligently to novel edge cases.

# Lab 03 Step 4 (REQ-0306): Selective Artifact Loading & Lab 03 Verification

Measuring progressive disclosure token savings and running `./labs/lab_03.../self_diagnose.sh`

## Selective Reference Loader (`select_references_for_artifact`)

- `artifact_type = "doc"`: Loads `["SKILL.md", "docs.md"]` (skips `slides-deck.md` & `apply_template.md`).
- `artifact_type = "slides"`: Loads `["SKILL.md", "slides-deck.md"]`.
- `artifact_type = "retrofit_doc"`: Loads `["SKILL.md", "docs.md", "apply_template.md"]`.
- Rejects unknown artifact types with `SkillSpecificationError`.

## Running Lab 03 Self-Diagnosis (`self_diagnose.sh`)

- **Command:**  
`./labs/lab_03_crafting_agent_skills_progressive_disclosure/self_diagnose.sh`
- **Verified Invariants (REQ-0301..0306):**
  - Valid `anthropic-brand/` passes all frontmatter, link, and signal gates.
  - `bad_self_generated_skill/` fails deterministically.
  - Broken relative links and chained multi-hop links are caught.

## Module 04 Complete → Next: Evaluating Skills (`eval-viewer`), Skill Overload & Trigger Calibration (Lab 04)

- Once we write a skill manually, how do we prove it actually helps ('eval-viewer') and prevent too many skills from confusing the router?

# Lab 04 Problem: Skill Catalog Overload & Uncalibrated Trigger Descriptions

Why adding  $\geq 4$  skills causes an 8.9 percentage-point drop and why raw Hit Rate is a trap

## Problem 1: Skill Count Overload (SkillsBench Table 8)

The More Skills You Add, The More Confused the Router Gets:

- **1 Skill: +18.0 pp** lift (36.6%  $\rightarrow$  54.6%).
- **2-3 Skills (Peak Sweet Spot): +19.0 pp** lift (32.2%  $\rightarrow$  51.2%).
- **$\geq 4$  Skills (Overload Collapse):** Drops to **+10.1 pp** (**-8.9 pp collapse** from peak!).
- Overlapping micro-skills (`git-commit`, `git-branch`, `git-pr`, `git-rebase`) compete in the startup system prompt.

## Problem 2: The Trigger “Trade-Off Trap”

Why Optimizing Raw Hit Rate Alone Fails:

- Broadening a skill's `description` raises Hit Rate from 85%  $\rightarrow$  100%, but False Alarms jump from 5%  $\rightarrow$  40%!
- Every False Alarm injects thousands of irrelevant Level 2 tokens into unrelated user tasks.
- We need **Signal Detection Theory** ( $d'$ ,  $c$ ) and paired `eval-viewer` benchmarking (`with_skill` vs. `without_skill`).

## The Reference-First Consolidation Rule (REQ-0404)

- Never create a standalone skill under **50 lines**. Merge micro-skills into `references/<subtopic>.md` inside one parent CUJ skill (**0 startup tokens, 0 extra routing targets**).

# Lab 04 Step 1 (REQ-0401..0402): Paired `with_skill` vs. `without_skill` Benchmarking

Computing `benchmark.json` deltas and enforcing the 11/11 zero-flake binomial reliability gate

## Anthropic `skill-creator` Paired Eval (`compute_paired_benchmark`)

- **Paired Runs:** Executes every eval prompt twice in parallel: `with_skill` and `without_skill` (baseline).
- **Per-Assertion Grading** (`grading.json`): Grades each expectation into `{text, passed, evidence}`.
- **Regression Detection:** Tracks `regressions_count` (assertions that passed in baseline but failed `with_skill`). Any regression fails the promotion gate!

## Why 3/3 Passes Is Not Enough: The 11/11 Rule

- **Binomial Reliability** ( $p^n \leq 1 - C$ ): To be  $C = 70\%$  confident that true pass rate is  $p \geq 0.90$ , how many consecutive passes  $n$  are required?
- At  $n = 3$ :  $0.90^3 = 0.729 \implies$  only **27%–34% confidence**!
- At  $n = 11$ :  $0.90^{11} = 0.313 \implies$  **72% statistical confidence** of  $\geq 90\%$  reliability.
- `promotion_approved` requires `mean_delta_pass_rate > 0` and `regressions_count == 0`.

## Starter Bug in `starter/skill_eval_engine.py`: Averaging `with_skill` Only

- The broken starter ignored the `without_skill` baseline completely—approving skills that actually caused regressions!

# Lab 04 Step 2 (REQ-0403..0404): Auditing Skill Catalog Overload

Enforcing the 2-3 active skill sweet spot and consolidating micro-skills into `references/*.md`

## Catalog Overload Detector (`audit_skill_catalog_overload`)

- **Rule 1 (Active Count > 3):** Flags `overload_detected = True` when more than `MAX_OPTIMAL_ACTIVE_SKILLS = 3` skills are active simultaneously (`SkillsBench` Table 8).
- **Rule 2 (Micro-Skills < 50 Lines):** Flags any standalone skill with `line_count < 50` for consolidation into `references/<name>.md`.
- **Rule 3 (Lexical Jaccard Overlap ≥ 0.35):** Computes token Jaccard similarity across skill descriptions to catch colliding triggers.

## Before vs. After Reference-First Consolidation

- **Before (Overloaded — 5 Skills, +10.1 pp Regime):**  
`git-commit-msg` (22L), `git-branch-name` (18L), `git-pr-template` (28L), `git-rebase-flow` (31L), `anthropic-brand` (95L).
- **After (Consolidated — 2 Skills, +19.0 pp Peak Regime):**  
`git-workflow/SKILL.md` (110L) + `references/{commit,branch,pr,rebase}.md` and `anthropic-brand/SKILL.md` (95L).

## Result: Zero Startup Token Increase & Zero Routing Ambiguity

- Moving 4 micro-skills into `references/*.md` under one CUJ parent cuts startup metadata tokens by 60% and restores the **+19.0 pp** peak lift.

# Lab 04 Step 3 (REQ-0405): Signal Detection Theory ( $d'$ , Criterion Bias $c$ , Utility $U$ )

Calibrating skill trigger descriptions mathematically with Hautus (1995) +0.5 smoothing

## Hautus (1995) Smoothed SDT Equations (`compute_sdt_metrics`)

- Given Hits ( $H$ ), Misses ( $M$ ), False Alarms ( $FA$ ), and Correct Rejections ( $CR$ ) on a 60/40 train/test split:

$$HR_{adj} = \frac{H + 0.5}{H + M + 1.0}, \quad FA_{adj} = \frac{FA + 0.5}{FA + CR + 1.0}$$

$$d' = \Phi^{-1}(HR_{adj}) - \Phi^{-1}(FA_{adj}), \quad c = -\frac{1}{2} [\Phi^{-1}(HR_{adj}) + \Phi^{-1}(FA_{adj})]$$

$$U = 2.0 \cdot d' - 0.5 \cdot |c - c_{target}|$$

## Interpreting $d'$ and Criterion Bias $c$

### • Sensitivity $d'$ (Discriminability):

- $d' > 2.0$ : Strong separation between positive and distractor prompts.
- $d' \leq 0$ : Trigger is no better than random coin tossing.

### • Criterion Bias $c$ (Threshold Placement):

- $c > +0.25$ : "conservative" (under-triggering / misses).
- $c < -0.25$ : "liberal" (over-triggering / false alarms).
- $|c| \leq 0.25$ : "balanced" (optimal operating point).

## Zero External Dependencies: Rational Acklam Inverse Normal $\Phi^{-1}(p)$ in Python `math`

- Lab 04 implements  $\Phi^{-1}(p)$  to  $< 1.15 \times 10^{-9}$  relative error using only Python's standard `math` module.

# Lab 04 Step 4 (REQ-0406): Rendering eval-viewer HTML & Lab 04 Verification

Generating the self-contained Outputs + Benchmark HTML review report

## `render_eval_viewer_html()` (REQ-0406)

- Mirrors Anthropic's `skills/skill-creator/eval-viewer/generate_review.py` HTML benchmark viewer:
- **Tab 1 (Outputs):** Side-by-side qualitative inspection of `with_skill` vs. `without_skill` outputs and per-assertion evidence.
- **Tab 2 (Benchmark):** Aggregate pass-rate lift (+45.0 pp), token delta, regressions count, SDT  $d'$ , bias  $c$ , and promotion badge.

## Running Lab 04 Self-Diagnosis (`self_diagnose.sh`)

- **Command:**  
`./labs/lab_04_skill_evaluation_and_trigger_calibration/self_diagnose.sh`
- **Verified Invariants (REQ-0401..0406):**
  - Paired benchmark blocks promotion when any baseline pass regresses.
  - $\geq 4$  skills and  $< 50$ -line micro-skills are flagged.
  - SDT  $d'$ ,  $c$ , and  $U$  rank balanced descriptions above noisy ones.

## Module 05 Complete → Next: Multi-Agent Collaboration with `feature-dev` (Lab 05)

- How do we coordinate specialized read-only subagents (`code-explorer`, `code-architect`, `code-reviewer`) across a 7-phase feature development workflow?

# Lab 05 Problem: Why Single-Window Vibe Coding Fails on Multi-File Features

How Anthropic's `feature-dev` plugin and Antigravity's Agent Manager isolate context across 7 phases

## Why Single-Session Monolithic Prompts Degrade

### 1. Search Pollution Pushes Context Past 60%:

- Running 25 raw grep/glob searches in the main agent thread fills 40,000 tokens before a single line of code is written.

### 2. Coding Before Clarifying (Missing Phase 3 Gate):

- Without a hard clarifying-questions gate, the agent guesses ambiguous edge cases and builds the wrong architecture.

### 3. Self-Review Confirmation Bias:

- An agent reviewing its own code in the same context window rubber-stamps its own bugs or floods the user with low-confidence nitpicks.

## Solution: The 7-Phase `feature-dev` Multi-Agent Pipeline

- **Phase 1:** Discovery & Todo Initialization.
- **Phase 2:** Parallel Read-Only `code-explorer` Subagents (`file:line`).
- **Phase 3: Mandatory Clarifying Questions Gate (STOP).**
- **Phase 4:** Tri-Lens `code-architect` (`minimal_changes`, `clean_architecture`, `pragmatic_balance`).
- **Phase 5:** Implementation & Test Verification.
- **Phase 6:** Parallel `code-reviewer` (`confidence >= 80` filter).
- **Phase 7:** Final Summary & Traceability Report.

## Subagent Isolation Principle: Subagents Explore Deeply, Return Compact Summaries (< 1,000 Tokens)

- Each `code-explorer`, `code-architect`, and `code-reviewer` runs in its own isolated context window with read-only tools (`Glob`, `Grep`, `Read`).

# Lab 05 Step 1 (REQ-0501..0502): Subagent Frontmatter & code-explorer

Validating read-only subagent definitions and merging parallel `file:line` exploration traces

## Subagent Frontmatter (`parse_agent_definition`)

- Stored in `.claude/agents/<name>.md` (`claude`) and `.agents/agents/<name>.md` (`agy`):

```
--  
name: code-explorer  
description: Traces execution paths with file:line citations  
tools: Glob, Grep, Read  
model: sonnet  
color: yellow  
--
```

- **Least-Privilege Guard (REQ-0501):** Rejects any explorer/architect/reviewer agent that requests write/shell tools (`Write`, `Edit`, `Bash`).

## Phase 2 Aggregation (`aggregate_exploration_reports`)

- Spawns 2–3 parallel `code-explorer` subagents (e.g. *entry-points* & *call-chains* vs. *domain invariants* & *tests*).
- **Citation Validator (REQ-0502):** Verifies every trace entry matches `^[a-zA-Z0-9_./-]+:[0-9]+$` (e.g. `src/ledger/batch.py:42`).
- Deduplicates `key_files_to_read` in deterministic order so the orchestrator reads only the essential files.

## Dual-Harness Execution: `/feature-dev` in Claude Code & Antigravity Agent Manager

- Both harnesses spawn the parallel read-only explorers concurrently and return structured `file:line` architectural maps to the parent session.

# Lab 05 Step 2 (REQ-0503): Phase 3 Mandatory Clarifying Questions Gate

Why Phase 4 Architecture and Phase 5 Implementation MUST raise `ClarificationGateError` until answered

## The Most Important Gate in `feature-dev`: Phase 3 STOP

- From Anthropic's official `plugins/feature-dev/commands/feature-dev.md`:  
“*CRITICAL: This is one of the most important phases. DO NOT SKIP. Wait for user answers before proceeding to architecture design.*”
- **What Phase 3 Synthesizes:** Edge cases, error-handling contracts, backward compatibility, and performance bounds discovered during Phase 2 exploration.

## Deterministic State-Machine Enforcement (REQ-0503)

- In `FeatureDevSession`, calling `run_phase4_architecture()` or `run_phase5_implementation()` while any question in `pending_questions` lacks a non-empty answer in `clarification_answers` raises `ClarificationGateError`!
- Only after `submit_clarifications(answers)` resolves every open question does the session transition to `Phase4_Architecture`.

## Antigravity Equivalent: Plan Mode Implementation Artifact Review & `.agents/workflows/feature-dev.md`

- In Antigravity, Phase 3 pauses in `/plan` mode or `/feature-dev` for explicit human sign-off on every underspecified requirement before writing code.

# Lab 05 Step 3 (REQ-0504..0505): Tri-Lens code-architect & Implementation

Comparing `minimal_changes`, `clean_architecture`, and `pragmatic_balance` before coding

## 1. `minimal_changes`

### Smallest Diff & Fastest Delivery:

- Reuses existing classes and call paths with minimal refactoring.
- Lowest blast radius for urgent hotfixes or small features.

## 2. `clean_architecture`

### Long-Term Maintainability:

- Introduces clean port/adaptor boundaries and decoupled interfaces.
- Best when building foundational domain modules.

## 3. `pragmatic_balance`

### Recommended Default:

- Balances delivery speed with clean interfaces and 100% unit testability.
- Human selects lens via `select_architecture_lens(lens)`.

## Phase 5 Implementation Gate (REQ-0505): Requires Both Approved Lens & Passing Verification

- `run_phase5_implementation()` raises `PhaseOrderError` if `selected_lens` is `None` or if `tests_passed=False`.

# Lab 05 Step 4 (REQ-0506): code-reviewer (confidence >= 80) & Verification

Filtering review noise and running `./labs/lab_05.../self_diagnose.sh`

## Phase 6 Quality Review (`filter_review_findings`)

- Spawns 3 parallel `code-reviewer` subagents auditing:
  1. *Simplicity / DRY / Elegance*
  2. *Functional Correctness & Edge-Case Bugs*
  3. *CLAUDE.md / GEMINI.md Convention Compliance.*
- **Strict Noise Filter (REQ-0506):** Suppresses any finding with `confidence < 80` (0..100 scale) and sorts high-confidence issues by severity (`critical` → `major`) and confidence descending.

## Running Lab 05 Self-Diagnosis (`self_diagnose.sh`)

- **Command:**  
`./labs/lab_05_multi_agent_feature_dev_orchestration/self_diagnose.sh`
- **Verified Invariants (REQ-0501..0506):**
  - Write tools (`Bash`, `Edit`) rejected on read-only subagents.
  - Missing `file:line` citations rejected.
  - Phase 3 `ClarificationGateError` blocks premature coding.
  - Findings with `confidence < 80` filtered out; Phase 7 summary generated.

## Module 06 Complete → Next: Versioned Plugins, MCP, Hooks & The Bridge to SDD (Lab 06 Capstone)

- How do we package our skills, subagents, hooks, and MCP servers into a versioned plugin—and when do we graduate to Spec-Driven Development?

# Lab 06 Problem: Skills vs. Versioned Plugins (plugin.json + MCP + Hooks)

When to use a standalone `SKILL.md` vs. a versioned plugin bundle across `claude` and `agy`

## Standalone Skill vs. Versioned Plugin Bundle

- **Standalone Skill** (`.claude/skills/<name>/` or `.agents/skills/<name>/`):
  - Pure Markdown instruction manual (`SKILL.md` + `references/`).
  - Invoked via `/skill-name`; ideal for project-local or single-workflow guidance.
- **Versioned Plugin** (`.claude-plugin/plugin.json` or `.agents/plugins/<name>/plugin.json`):
  - SemVer package (`1.2.0`) bundling `skills/`, `agents/`, `commands/` or `rules/`, `hooks/hooks.json`, and **MCP servers** (`.mcp.json` / `mcp_config.json`).
  - Namespaced (`/plugin-name:skill-name`) for org-wide distribution.

## Anatomy of `vibe-engineering-kit` (v1.2.0 in Lab 06)

```
vibe-engineering-plugin/  
|- .claude-plugin/plugin.json  (& .agents/plugins/.../plugin.json)  
|- .mcp.json  (ledger-schema-mcp stdio JSON-RPC server)  
|- hooks/hooks.json  (PreToolUse immutable-verifier guard)  
|- skills/anthropic-brand/SKILL.md  
|- agents/{code-explorer,code-architect,code-reviewer}.md  
`- commands/feature-dev.md
```

## Why Plugins Bundle Skills + MCP Together

- **MCP Servers** give the agent deterministic *hands and eyes* (live schema/API tools); **Skills** give the agent the *procedural brain* explaining when and how to call those tools.

# Lab 06 Step 1 (REQ-0601..0602): Validating plugin.json & Bundled MCP Servers

Enforcing SemVer manifests and stdio JSON-RPC MCP tool contracts in `harness_flywheel.py`

## Plugin Manifest Validator (`validate_plugin_bundle`)

- **Kebab-Case Name (REQ-0601):** Validates `^[a-z0-9]+(-[a-z0-9]+)*$` (e.g. `vibe-engineering-kit`).
- **Strict SemVer Version:** Validates `^\d+\.\d+\.\d+$` (e.g. `1.2.0`); rejects `"latest"` or unversioned bundles.
- **Component Resolution:** Verifies that every skill, subagent (`.md`), command, hook config, and MCP server referenced by the plugin exists on disk.

## Bundled MCP Server (`.mcp.json` / `mcp_config.json`)

- **Validation (REQ-0602):** Ensures every MCP server defines a non-empty `command`, `args` list, and `tools` schema.
- **Included Reference Server (`mcp_ledger_server.py`):** Implements a zero-dependency stdio JSON-RPC 2.0 server exposing `query_ubiquitous_Show_schema` and `check_fx_rate_bps` so agents query live domain truth instead of guessing.

## Starter Bug in `starter/harness_flywheel.py`: Accepting Unversioned & Broken Bundles

- The broken starter accepted `version: "latest"`, missing subagent files, and empty MCP server definitions without raising `PluginValidationError`.

# Lab 06 Step 2 (REQ-0603): Deterministic PreToolUse Guardrail Hooks

Blocking sycophantic self-testing by failing closed on `adversarial_tests/` and `chmod -w`

## Why Prompt Rules Alone Cannot Protect Verifiers

- Under heavy pressure to make a failing test suite pass, an unconstrained coding agent may edit the test assertion (`assert False -> assert True`) or run `chmod +w adversarial_tests/`.
- Advisory instructions in `CLAUDE.md` or `GEMINI.md` are probabilistic; security and verifier integrity must be **deterministic**.

## `evaluate_pre_tool_use_hook()` (REQ-0603)

- Intercepts `Write`, `Edit`, `MultiEdit`, `Bash`, and `run_command` before execution:
  1. Blocks any write/edit targeting `adversarial_tests/`, `self_diagnose.sh`, or `.git/`.
  2. Blocks shell commands containing `chmod`, `rm -rf`, `git push -force`, or `adversarial_tests`.
- Returns `{"decision": "block", "exit_code": 2, "reason": "..."}.`

## Architectural Rule: Prompts Guide Behavior; PreToolUse Hooks Enforce Hard Boundaries

- Both `.claude/settings.json` and `.agents/plugins/vibe-engineering-kit/hooks/hooks.json` wire `pre_tool_use_guard.py` into the live harness.

# Lab 06 Step 3 (REQ-0604..0605): The “On-the-Loop” Harness Flywheel Router

Routing every observed agent failure to the right harness layer in `recommend_flywheel_upgrade()`

## 1. ROOT\_CONTEXT\_RULE

**Trigger:** `wrong_test_command`

- Agent runs wrong build/test CLI.
- **Fix:** Add 1-line command to top of `CLAUDE.md` / `GEMINI.md`.

## 2. AUTO\_MEMORY

**Trigger:** `repeated_user_correction`

- User repeats a preference across turns.
- **Fix:** Persist to `MEMORY.md` / `KNOWLEDGE.md` with `#direct`.

## 3. CURATED\_SKILL

**Trigger:** `domain_procedure_error`

- Repeatable workflow/gotcha failure.
- **Fix:** Encode Delta in `SKILL.md` & verify in `eval-viewer`.

## 4. PRE\_TOOL\_USE\_HOOK

**Trigger:** `modified_locked_tests`

- Agent attempts to edit locked tests.
- **Fix:** Block deterministically in `hooks.json` (`exit_code: 2`).

## 5. PLUGIN\_MCP\_TOOL

**Trigger:** `missing_live_schema_tool`

- Agent guesses live database/FX state.
- **Fix:** Expose deterministic JSON-RPC tool in `.mcp.json`.

## 6. ESCALATE\_TO\_SDD\_SPEC

**Trigger:**  $\geq 3$  services or breaking API

- Conversational vibe coding hits its ceiling.
- **Fix:** Escalate to `SDD-Crash-Course!`

# Lab 06 Step 4 (REQ-0606): The Escalation Bridge to SDD-Crash-Course

Knowing exactly when to graduate from Disciplined Vibe Coding to Spec-Driven Development

## When Disciplined Vibe Coding Suffices

- Single-service features, UI prototypes, self-contained utilities, and bounded refactorings (1..2 services, no breaking wire/schema contracts).
- Governed by our 5 Vibe-Coding Harnesses: Ubiquitous Language types (Lab 01),  $\leq$  40-line root context + Auto-Memory (Lab 02), curated skills (Lab 03), `eval-viewer` + SDT  $d'$  (Lab 04), and `feature-dev` subagents (Lab 05).

## When to Escalate to SDD-Crash-Course (REQ-0606)

- When `services_touched >= 3` OR `has_breaking_schema_or_api_change=True` OR `requires_clean_room_rebuild_ssot=True`:
  1. **Conductor** (`conductor/tracks/<id>/`): Context-driven `spec.md` + phased TDD `plan.md`.
  2. **OpenSpec** (`openspec/changes/<id>/`): Normative RFC 2119 + Gherkin capability deltas.
  3. **SPEC.md (SSOT)**: 8-section Clean-Room Rebuild blueprint.

## Course Complete! Run `./self_diagnose_all.sh` (6/6 Labs, 54 Tests in $< 0.1s$ ) & Proceed to SDD-Crash-Course

- `git checkout agy` (Antigravity Track) | `git checkout claude` (Claude Code Track) | `./self_diagnose_all.sh` → **[ALL PASSED] 6/6 Labs Verified!**